

Project Two Template

MAT-350: Applied Linear Algebra

Kenneth Greer Jr

12-14-2025

Problem 1

Use the `svd()` function in MATLAB to compute A_1 , the **rank-1 approximation** of A . Clearly state what A_1 is, rounded to 4 decimal places. Also, **compute** the root-mean square error (RMSE) between A and A_1 .

Solution:

```
A = [1 2 3;
      3 3 4;
      5 6 7];
[U,S,V] = svd(A);
A1 = U(:,1) * S(1,1) * V(:,1).';
sigma1 = S(1,1);
err_norm = norm(A - A1, 'fro');
RMSE1 = err_norm / sqrt(numel(A));
sigma1
```

```
sigma1 =
12.5318
```

```
RMSE1
```

```
RMSE1 =
0.3257
```

Problem 2

Use the `svd()` function in MATLAB to compute A_2 , the **rank-2 approximation** of A . Clearly state what A_2 is, rounded to 4 decimal places. Also, **compute** the root-mean square error (RMSE) between A and A_2 . Which approximation is better, A_1 or A_2 ? Explain.

Solution:

```
A2 = U(:,1:2) * S(1:2,1:2) * V(:,1:2).';
err_norm = norm(A - A2, 'fro');
RMSE2 = err_norm / sqrt(numel(A));
RMSE2
```

```
RMSE2 =
0.1166
```

Explain: The rank 2 approximation is better than the rank 1 approximation because it includes an additional singular value and corresponding singular vectors which allows it to capture more information from the original matrix. This results in a smaller RMSE which indicates a more accurate approximation.

Problem 3

For the 3×3 matrix A , the singular value decomposition is $A = USV'$ where $U = [\mathbf{u}_1 \ \mathbf{u}_2 \ \mathbf{u}_3]$. Use MATLAB to **compute** the dot product $d_1 = \text{dot}(\mathbf{u}_1, \mathbf{u}_2)$.

Also, use MATLAB to **compute** the cross product $\mathbf{c} = \text{cross}(\mathbf{u}_1, \mathbf{u}_2)$ and dot product $d_2 = \text{dot}(\mathbf{c}, \mathbf{u}_3)$. Clearly state the values for each of these computations. Do these values make sense? **Explain.**

Solution:

```
u1 = [-0.2055; -0.4900; -0.8471];  
u2 = [-0.6658; -0.5644;  0.4880];  
u3 = [-0.7172;  0.6643; -0.2103];  
d1 = dot(u1,u2);  
c = cross(u1,u2);  
d2 = dot(c,u3);  
d1
```

```
d1 =  
-6.9000e-06
```

```
c
```

```
c = 3x1  
 -0.7172  
  0.6643  
 -0.2103
```

```
d2
```

```
d2 =  
0.9999
```

Explain: Since $U = [\mathbf{u}_1 \ \mathbf{u}_2 \ \mathbf{u}_3]$ is an orthogonal matrix, the dot product of $\mathbf{u}_1$ and $\mathbf{u}_2$ should be 0. d_1 's answer is approximately 0 so that checks out. The dot product of two vector $\mathbf{c}$ and $\mathbf{u}_3$ should be 1, and it is approximately 1. These results confirm that the singular vectors form a right handed orthonormal basis, which is a fundamental geometric property of orthogonal matrices that come from SVD.

Problem 4

Using the matrix $U = [\mathbf{u}_1 \ \mathbf{u}_2 \ \mathbf{u}_3]$, determine whether or not the columns of U span $\mathbb{R}^3$. Explain your approach.

Solution:

```
rankU = rank(U);  
n = size(U,1);  
rankU
```

```
rankU =  
3
```

```
n
```

```
n =  
3
```

Explain: The columns of a matrix span $\mathbb{R}^n$ if the rank of the matrix equals n . Since the rank of U is 3 and U has 3 rows, the columns of U span $\mathbb{R}^3$. This makes sense because U is an orthogonal matrix whose columns form an orthonormal basis. Because the columns of an orthogonal matrix are mutually perpendicular unit vectors, they not only span $\mathbb{R}^3$, but also form an optimal basis for the space.

Problem 5

Use the MATLAB `imshow()` function to load and display the image A stored in the `image.mat` file, available in the Project Two Supported Materials area in Brightspace. For the loaded image, **derive the value of k** that will result in a compression ratio of $CR \approx 2$. For this value of k , **construct the rank- k approximation of the image**.

Solution:

```
load('MAT 350 Project Two MATLAB Image.mat')  
imshow(A)
```



```
CR = 2
```

```
CR =  
2
```

```
[m,n] = size(A);  
k = floor((m*n)/(CR*(m+n+1)));  
[U, S, V] = svd(double(A))
```

```
U = 2583x2583  
 0.0106    0.0360    0.0006   -0.0032    0.0032   -0.0041    0.0066   -0.0022 ...  
 0.0105    0.0361    0.0006   -0.0030    0.0035   -0.0049    0.0062   -0.0020 ...  
 0.0105    0.0362    0.0006   -0.0034    0.0037   -0.0042    0.0064   -0.0025 ...  
 0.0105    0.0362    0.0009   -0.0029    0.0035   -0.0052    0.0056   -0.0028 ...  
 0.0106    0.0361    0.0011   -0.0034    0.0035   -0.0046    0.0061   -0.0022 ...  
 0.0106    0.0363    0.0011   -0.0031    0.0030   -0.0049    0.0061   -0.0031 ...  
 0.0106    0.0364    0.0008   -0.0032    0.0032   -0.0043    0.0057   -0.0033 ...  
 0.0106    0.0365    0.0006   -0.0029    0.0033   -0.0050    0.0052   -0.0031 ...  
 0.0106    0.0366    0.0007   -0.0033    0.0031   -0.0040    0.0053   -0.0033 ...  
 0.0106    0.0368    0.0009   -0.0030    0.0034   -0.0044    0.0052   -0.0032 ...  
 0.0106    0.0367    0.0009   -0.0029    0.0034   -0.0044    0.0051   -0.0036 ...  
 0.0106    0.0365    0.0009   -0.0030    0.0031   -0.0047    0.0049   -0.0037 ...  
 0.0106    0.0367    0.0007   -0.0034    0.0030   -0.0047    0.0045   -0.0028 ...  
 0.0106    0.0367    0.0004   -0.0035    0.0036   -0.0048    0.0050   -0.0038 ...  
 0.0106    0.0369    0.0003   -0.0030    0.0032   -0.0041    0.0047   -0.0039 ...  
  :
```

```
S = 2583x4220  
105 ×  
 4.0600         0         0         0         0         0         0         0 ...
```

0	0.8702	0	0	0	0	0	0
0	0	0.4169	0	0	0	0	0
0	0	0	0.4104	0	0	0	0
0	0	0	0	0.3405	0	0	0
0	0	0	0	0	0.2992	0	0
0	0	0	0	0	0	0.2550	0
0	0	0	0	0	0	0	0.2268
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

⋮

V = 4220x4220

0.0130	-0.0044	0.0358	-0.0028	0.0085	-0.0177	-0.0128	0.0163	...
0.0130	-0.0045	0.0357	-0.0024	0.0079	-0.0184	-0.0134	0.0162	
0.0130	-0.0045	0.0359	-0.0025	0.0078	-0.0181	-0.0124	0.0168	
0.0130	-0.0046	0.0361	-0.0030	0.0087	-0.0185	-0.0125	0.0158	
0.0130	-0.0045	0.0366	-0.0032	0.0095	-0.0182	-0.0116	0.0149	
0.0129	-0.0046	0.0369	-0.0034	0.0095	-0.0185	-0.0112	0.0151	
0.0130	-0.0047	0.0372	-0.0046	0.0104	-0.0178	-0.0103	0.0141	
0.0130	-0.0048	0.0377	-0.0045	0.0097	-0.0179	-0.0108	0.0145	
0.0130	-0.0046	0.0375	-0.0049	0.0094	-0.0170	-0.0102	0.0147	
0.0130	-0.0045	0.0379	-0.0043	0.0090	-0.0166	-0.0095	0.0142	
0.0130	-0.0042	0.0382	-0.0045	0.0090	-0.0162	-0.0094	0.0135	
0.0130	-0.0042	0.0383	-0.0041	0.0085	-0.0161	-0.0100	0.0141	
0.0129	-0.0039	0.0380	-0.0043	0.0083	-0.0165	-0.0096	0.0133	
0.0129	-0.0035	0.0382	-0.0037	0.0080	-0.0162	-0.0095	0.0135	
0.0128	-0.0032	0.0386	-0.0037	0.0073	-0.0158	-0.0099	0.0122	

⋮

Ak = U(:,1:k) * S(1:k, 1:k) * V(:,1:k).'

Ak = 2583x4220

26.4896	27.2541	30.5810	28.9530	23.3828	25.7705	35.1037	29.3968	...
32.6831	34.0733	28.4258	30.5682	27.6882	28.4547	35.6629	31.2002	
35.5230	30.8250	18.7994	19.9743	19.8952	17.0400	24.6764	26.0911	
33.6440	29.7702	26.1173	29.8858	26.5095	15.9739	24.7017	25.8886	
27.7165	26.0012	30.5018	36.7547	35.3434	29.8915	34.5078	25.1416	
27.3996	25.5183	26.6092	29.4463	25.5795	28.8615	33.9147	23.0624	
32.0484	32.4889	27.5089	22.7250	20.3684	25.0803	33.3388	26.7700	
26.0954	32.2044	27.4183	18.1894	21.2836	28.1417	31.7244	26.2813	
23.2187	25.5412	22.1689	25.1362	29.2165	30.3222	34.5845	30.5812	
21.3048	20.9797	19.1568	25.5860	26.9030	24.8220	30.0068	30.1700	
21.0570	20.5597	20.4413	28.6261	24.3885	24.3385	26.3504	21.4459	
22.1100	23.3674	24.9443	30.5527	21.3032	17.7011	22.2625	23.0917	
27.2234	29.9587	25.7000	27.7856	24.1104	21.8076	25.1441	25.7437	
26.3002	27.8818	23.8573	29.6187	30.6100	28.4090	26.3002	21.7899	
23.0872	23.2337	20.9303	25.9354	26.7234	24.2094	26.1100	20.1887	

⋮

Ak = uint8(round(Ak))

Ak = 2583x4220 uint8 matrix

26	27	31	29	23	26	35	29	33	36	30	26	32	27	28	34	...
33	34	28	31	28	28	36	31	30	27	22	28	40	34	34	34	
36	31	19	20	20	17	25	26	30	23	22	27	35	35	37	34	
34	30	26	30	27	16	25	26	28	25	22	24	33	35	32	30	

```

28  26  31  37  35  30  35  25  25  25  23  27  33  33  27  27
27  26  27  29  26  29  34  23  22  27  29  34  33  33  28  26
32  32  28  23  20  25  33  27  27  29  35  33  27  28  31  29
26  32  27  18  21  28  32  26  29  32  36  31  28  31  34  34
23  26  22  25  29  30  35  31  30  35  34  27  20  24  34  34
21  21  19  26  27  25  30  30  28  34  29  20  16  22  29  32
21  21  20  29  24  24  26  21  25  29  25  25  25  24  30  29
22  23  25  31  21  18  22  23  29  24  20  24  27  27  32  27
27  30  26  28  24  22  25  26  36  29  20  20  20  24  32  24
26  28  24  30  31  28  26  22  31  30  24  22  22  25  31  24
23  23  21  26  27  24  26  20  23  24  27  29  29  26  23  23
:
:

```

```
k
```

```
k =
801
```

Explain: The value of k is computed using the compression ratio formula provided in the project overview. The rank k approximation keeps the most significant singular values while achieving the compression.

Problem 6

Display the image and compute the root mean square error (RMSE) between the approximation and the original image. Make sure to include a copy of the approximate image in your report.

Solution:

```
figure
imshow(Ak)
```



```
err_norm = norm(double(A) - double(Ak), 'fro');
RMSEk = err_norm / sqrt(m*n);
RMSEk
```

```
RMSEk =
3.1664
```

Problem 7

Repeat Problems 5 and 6 for $CR \approx 10$, $CR \approx 25$, and $CR \approx 75$. **Explain** what trends you observe in the image approximation as CR increases and provide your recommendation for the best CR based on your observations. Make sure to include a copy of the approximate images in your report.

Solution:

```
CR = 10;
k = floor((m*n)/(CR*(m+n+1)));
Ak = U(:,1:k) * S(1:k,1:k) * V(:,1:k).';
Ak = uint8(round(Ak));
figure
imshow(Ak)
```




```
err_norm = norm(double(A) - double(Ak), 'fro');  
RMSE10 = err_norm / sqrt(m*n);  
  
CR = 25;  
k = floor((m*n)/(CR*(m+n+1)));  
Ak = U(:,1:k) * S(1:k,1:k) * V(:,1:k).';  
Ak = uint8(round(Ak));  
figure  
imshow(Ak)
```




```
err_norm = norm(double(A) - double(Ak), 'fro');  
RMSE25 = err_norm / sqrt(m*n);  
  
CR = 75;  
k = floor((m*n)/(CR*(m+n+1)));  
Ak = U(:,1:k) * S(1:k,1:k) * V(:,1:k).';  
Ak = uint8(round(Ak));  
figure  
imshow(Ak)
```



```
err_norm = norm(double(A) - double(Ak), 'fro');  
RMSE75 = err_norm / sqrt(m*n);
```

Explain:

As the compression ratio decreases, the value of k increases, resulting in improved image quality and a smaller RMSE. Lower compression ratios retain more singular values and preserve the finer details, while higher compression ratios discard more information and increase errors. This trade off of quality and compression leads me to believe a middle ground would be best to balance compression and image quality.